How It Works

A learning guide to **Emergent Communication in a Grid World**

Christian Tadros · CAP6938 AI Knowledge Systems

What's inside

0. The project in one page
1. The ideas you need (no background assumed)
2. The world the agents live in
3. The code, piece by piece
4. How the experiment was run
5. Reading the results
6. The vocal channel (bonus experiment)
7. v2: words that combine
8. Try it yourself
9. Glossary
10. Questions your professor might ask

> **How to read this guide.** Part 0 is the whole story in one page. Parts 1 and 2 explain the ideas and the world without any code. Part 3 opens the hood: every file in `src/emcomm/`, what it does, and how the pieces connect. Parts 4 and 5 explain how the experiment was run and how to read each figure. Part 10 is a cheat sheet for questions. Every result number in this guide is filled in automatically from generated result files, the same sources the report uses, so the guide and the report always agree.

0. The project in one page

The question. Can two AI agents that learn on their own invent a language to coordinate, when nobody ever tells them what any word means?

The game. Two agents live on a 12×12 grid split by a solid wall.

- The **scout** sits in a small *control room*. It is the only one told which of four items (A, B, C, D) is wanted. It can never reach an item.
- The **runner** sits in the *warehouse* with all four items and a drop-off spot (the *depot*). It can pick up and deliver items, but it is never told which one is wanted, and it only sees a 7×7 window around itself.
- They cannot see each other. Their only connection is a radio: every step, each one sends one of 8 "symbols" (think of them as 8 different beeps), and the other hears it one step later. **The symbols start with no meaning.**
- Delivering the right item wins the round. Delivering a wrong item loses it. So a runner who just guesses wins about 1 time in 4.

The training. Each pair of agents practiced for 10.0 million environment steps, receiving shared task rewards and the disclosed signalling objective. Twenty pairs were trained from scratch for each of four setups, 80 training runs in total.

The result. With the radio on, the agents won **87.6%** of rounds on maps they had never seen, against **22.5%** with the radio off. When the meaning of their symbols was secretly scrambled, success collapsed, which supports dependence on stable symbol identities. Symbol assignments were learned rather than supplied. The pre-registered success criterion from the proposal was met.

The training choice. A small pilot favored an auxiliary objective that makes the scout's messages informative about its cue. That is a task-specific adaptation of prior positive-signalling work, not evidence that plain IPPO can never communicate. The full-budget direct ablation is separately labeled exploratory; see Part 4.3.

1. The ideas you need

1.1 Agents, environments, and rewards

Reinforcement learning (RL) is learning by trial and error. There are three ingredients:

- An **environment**: the world, with rules. Here, the grid.
- An **agent**: something that looks at the world, picks an action, and repeats. Here, the scout and the runner.
- A **reward**: a number the environment hands out after each action. Here, +1 for delivering the right item.

The agent's only goal is to collect as much reward as possible. Nobody shows it the right moves. It tries things, sees what earned reward, and gradually does more of what worked.

A few words used throughout:

Word	Meaning here
Step	One tick of the clock. Both agents act once (a move and a symbol).
Episode (or round, or game)	One full attempt, from the start until a delivery or until 32 steps run out.
Observation	What an agent can perceive at one step: 315 numbers (Part 2.3).
Action	What an agent does at one step: a move plus a symbol (Part 2.4).
Policy	The agent's decision rule: "given what I see and remember, how likely am I to take each action?"

1.2 A policy is built from a neural network

A **neural network** is a big adjustable formula. Numbers go in (the observation), numbers come out (how likely each action is). Inside are about 158,000 adjustable numbers called **weights** for each agent. A freshly created network has random weights, so it acts randomly. **Training means adjusting the weights** until the network's choices earn reward.

The network does not output one action. It outputs a *probability* for each action ("go up 60%, go left 25%, …"), and the action is drawn at random from those probabilities. That randomness is how the agent explores.

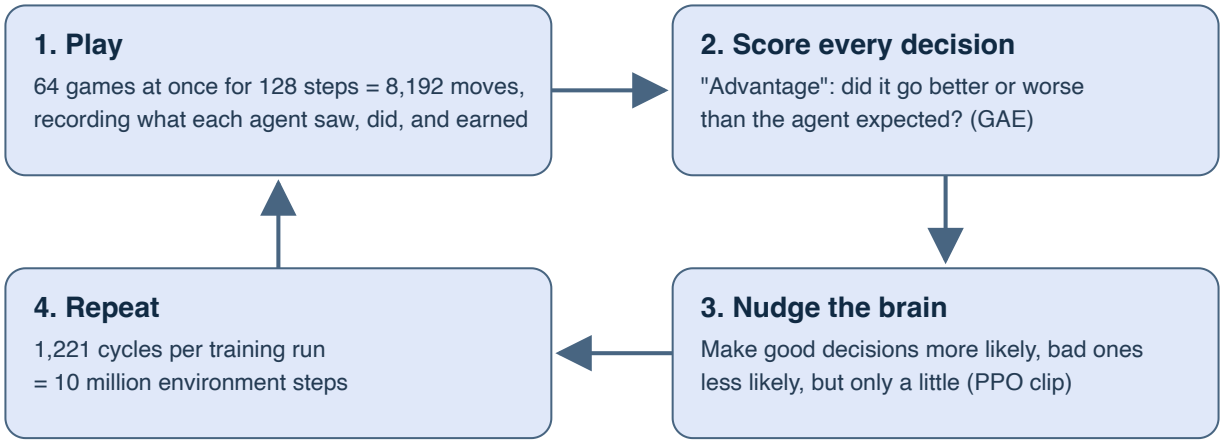
1.3 How a network learns, in one paragraph

After the agents play for a while, we look at every decision they made and ask whether it turned out *better or worse than expected*. For each weight, calculus tells us which direction to change it so that better-than-expected decisions become a bit more likely and worse-than-expected ones a bit less likely. We move every weight a tiny step in that direction. This is **gradient descent**, done with an optimizer

called **Adam** and a step size (**learning rate**) of 0.001. Repeat this thousands of times and the behavior improves.

1.4 PPO: the specific learning recipe

The project uses **PPO** (Proximal Policy Optimization), one of the most widely used RL recipes. One cycle looks like this:



The three ideas inside PPO, in plain terms:

- **The value estimate.** Besides choosing actions, the network also predicts "how much reward do I expect from here?" That prediction is the *baseline*. An action is good if the result beat the baseline, bad if it fell short. The difference is the **advantage**.
- **GAE** (Generalized Advantage Estimation) is a standard way to compute advantages that blends short-term and long-term outcomes. Two settings control it: $\gamma = 0.99$ (future reward counts almost as much as immediate reward) and $\lambda = 0.95$.
- **Clipping.** PPO limits the training incentive for already-large probability-ratio changes (clip = 0.2). This encourages cautious updates; it does not impose a hard 20% bound on the final probabilities.

PPO also adds a small **entropy bonus** (0.01) that rewards keeping some randomness, so the agents keep exploring instead of locking into their first habits.

1.5 Memory: the GRU

The runner hears the scout's symbol and must keep acting on it for many steps while it searches the warehouse. That needs **memory**. Each agent's network contains a **GRU** (Gated Recurrent Unit): a block of 128 numbers that is updated every step from the new observation and carried forward to the next step. It is the agent's short-term memory. At the start of every new game it is wiped to zeros.

1.6 Two learners, each on its own

Each agent has its **own separate network** and learns only from its own experience. They share only the team reward. There is no shared brain and no central coach that sees everything. This setup is called **IPPO** (Independent PPO). It is what makes the proposal's phrase "independently learning agents" literally true: the policies learn separately from local inputs and shared task reward, with the declared cue objective added where a cue is visible.

1.7 Emergent communication, and why it is hard

Emergent communication means learned message assignments and use, rather than a programmed dictionary. This project also uses a disclosed auxiliary signalling objective. It runs into a **chicken-and-egg problem**:

- The scout gains nothing from making its symbols meaningful until the runner pays attention to them.
- The runner gains nothing from paying attention until the symbols mean something.

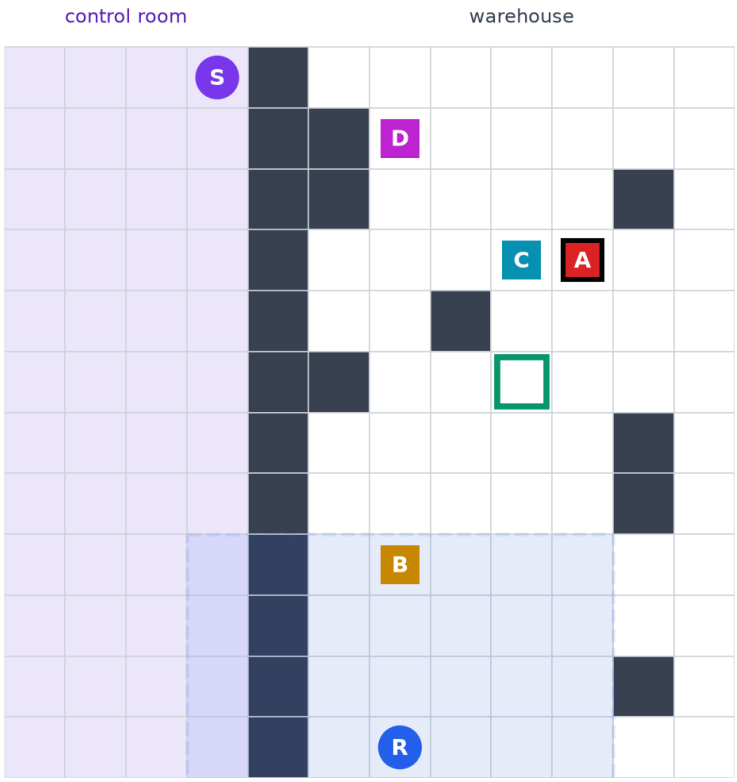
So at the start, neither has a reason to move first. That is exactly what happened in this project's pilot: with plain IPPO, the scout's symbols drifted into random habits that carried almost no information, the runner learned to ignore them, and success stayed at guessing level.

The fix: a cue-informativeness bias. The scout gets an auxiliary training objective for making its symbol choices *depend on the target it was told*. Mathematically this is the **mutual information** between its symbols and its cue: how much knowing the symbol tells you about the target (maximum 2 bits for 4 targets). This bias is a targeted version of the "positive signalling" bias from published research (Eccles et al., 2019). Crucially, it never says *which* symbol should mean *which* item, and the runner gets no supervised decoder or listener-specific loss. Symbol assignments develop under the added cue objective; the runner learns their use through task reward.

> **In plain words:** the bias tells the scout "don't mumble; say *something* different for each item." It does not tell it what to say, and it does not teach the runner anything.

2. The world the agents live in

2.1 The map



A real held-out episode, drawn from the environment code. Purple: the control room with the scout (S). Dark squares: walls and obstacles. The four colored items are A–D; the target (A) has a black outline. The green square is the depot. The dashed blue box is the runner's (R) 7×7 field of view.

- **Size:** 12×12 cells. Column 4 is a solid wall with no door.
- **Control room:** columns 0–3, scout only.
- **Warehouse:** columns 5–11, with the runner, the four items, the depot, and 8 obstacle cells.
- **Perception:** views are square crops, without wall occlusion. A v1/vocal scout may see nearby warehouse content across the wall; v2 masks that content explicitly. Neither agent sees the other agent's position.
- **Layouts:** 72 different obstacle arrangements were generated from a fixed seed. **48 are used for training** and **24 are held out**: the agents never see them until the final test. Testing on held-out layouts measures transfer within this layout generator; it does not rule out every kind of overfitting.

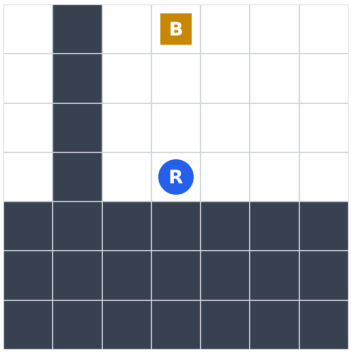
2.2 One episode

1. A layout is picked, and the four items, the depot, the runner, and the scout are placed at random.
2. A target item is picked at random. Only the scout's observation includes it.

3. Each step, both agents act at the same time: each picks a move and sends a symbol.
4. The episode ends when the runner delivers any item (right = success, wrong = failure) or after **32 steps** (failure).

2.3 What each agent observes: 315 numbers

Block	Size	What it is
Local view	294	A 7×7 window around the agent, with 6 layers: walls (off-map counts as wall), item A, item B, item C, item D, depot.
Position	2	Its own row and column, scaled to 0–1.
Carrying	5	Nothing, A, B, C, or D (one switch on).
Target cue	4	Which item is wanted. **All zeros for the runner** (except in the oracle setup).
Heard symbol	9	The symbol it heard this step, 0–7, or "silence".
Time	1	Steps used so far, divided by 32.



What the runner actually perceives at the start of the episode above, decoded from its real observation vector. It can see item B, the wrong one, and the rows below it are off the map (shown as wall). Without a message, it has no idea that A is the target.

2.4 What each agent can do

Every step, each agent chooses **two things at once**:

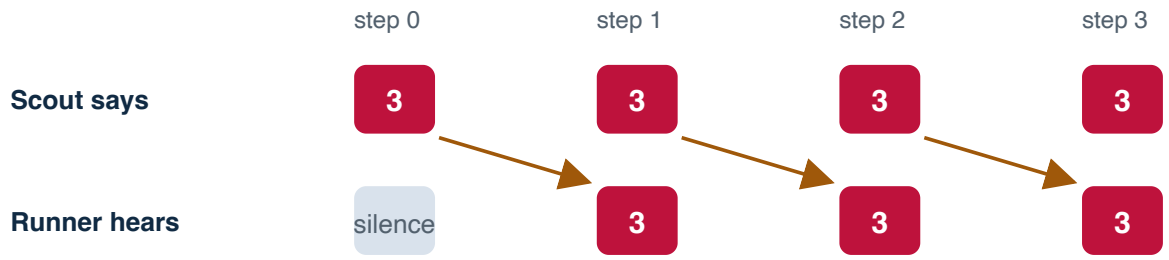
1. **A move:** stay, up, down, left, right, or *interact*. Interact picks up an item if the agent is standing on one and carrying nothing, or delivers the carried item if it is standing on the depot.
2. **A symbol:** one of 8, numbered 0–7.

Both agents have exactly the same choices. The scout's moves are mostly pointless (it can never reach an item), but keeping the action sets identical keeps the comparison clean.

2.5 Rewards

Event	Reward (shared by both agents)
Every step	−0.01 (a small push to be quick)
Picking up any item	+0.1
Delivering the target	+1, episode ends: **success**
Delivering a wrong item	0, episode ends: **failure**
Running out of 32 steps	episode ends: **failure**

2.6 The radio channel



What one agent sends at step *t*, the other hears at step *t* + *1*. At step 0 nobody has spoken yet, so both hear "silence". The channel runs both ways (the runner also sends symbols), but only the scout has anything useful to say.

2.7 Three tricks that make the test fair

1. **The scout can't do it alone.** The wall is solid, so the scout can never fetch an item itself. The target information *must* travel through the radio.
2. **No signalling by movement.** The agents' observations simply do not contain each other, so the scout can't "point" by walking somewhere. The radio is the only channel.
3. **The map can't give away the answer.** An episode is only accepted if *every* item could be delivered in time (within 20 steps by a runner who knows the target), and the target is chosen only *after* the map is accepted. If only the target had to be close, the target would usually be the nearest item, and a guessing runner could win by grabbing the closest one. With this rule the nearest item is right only 1 time in 4.

The project checked all three with a hand-written **scripted team** that plans on the true map. With a fixed codebook it succeeds 100.0% of the time; with no communication only 26.5%, which is chance. This supports high-success coordination with communication; an uncued runner can still guess successfully in some episodes. This scripted team is only a reference, never a competitor: it cheats by knowing the whole map.

2.8 Four setups and three tests

Every setup uses identical networks, settings, spaces and training length. The partial-information controls differ in radio content; the oracle also reveals the target cue:

Setup	Runner told the target?	What the runner hears
Learned comm	no	the scout's last symbol
No comm	no	always "silence"
Random messages	no	a random symbol every step
Oracle	**yes**	always "silence"

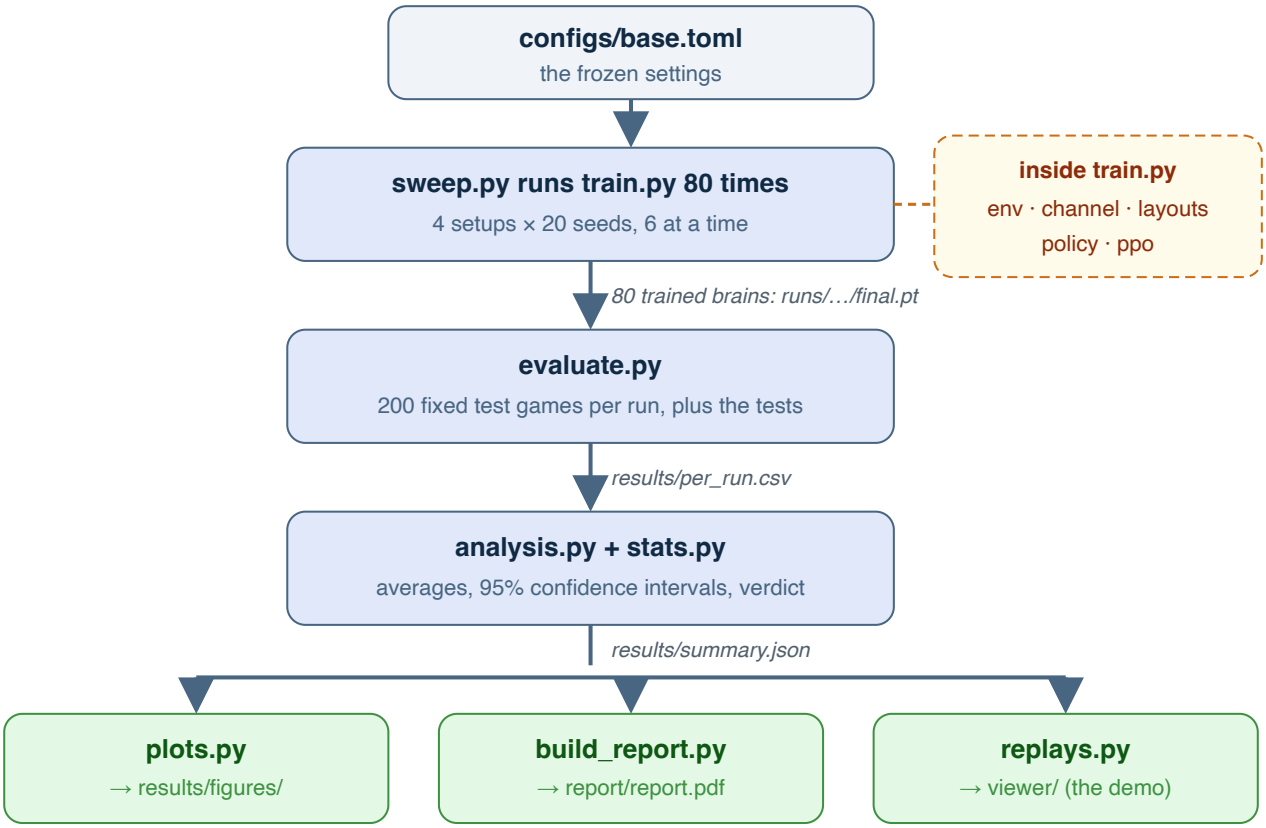
- **No comm** shows what guessing achieves.
- **Random messages** checks that it's *information* that helps, not just having extra input.
- **Oracle** is a learned full-information reference for navigation performance, not a mathematical upper bound.

After training, the learned-comm pairs were also tested under three conditions:

- **Intact:** normal.
- **Permuted:** in each test game, the 8 symbols are relabeled at random before the runner hears them (for example, 3 arrives as 7). If the runner reads meanings, this should destroy its success.
- **Silenced:** the runner hears nothing.

3. The code, piece by piece

The v1 baseline had about 1,800 lines of Python in `src/emcomm/` and 66 automated tests in `tests/`. The v2 extension adds modules and checks. Here is how the baseline pieces fit together, from settings file to report:



The code is organized in four layers. The lower layers know nothing about the upper ones.

3.1 Layer 1: the world (no learning)

`config.py` **(the settings).** Two frozen "dataclasses" hold every setting: `EnvConfig` (grid size 12, wall at column 4, view radius 3, 4 item types, 32-step limit, 8 symbols, rewards…) and `PPOConfig` (64 parallel games, 128-step rollouts, learning rate 0.001, the bias weight 0.2…). `load_config()` reads `configs/base.toml` and refuses unknown keys, so a typo can't silently be ignored. It also computes derived facts, such as the observation size (315) and where the target cue sits inside it.

`layouts.py` **(maps and episodes).**

- `make_layout_splits()` generates the 72 obstacle maps from a fixed seed and splits them 48/24.
- `bfs_distances()` computes the shortest walking distance from a cell to every other cell (breadth-first search), avoiding walls.
- `try_geometry()` places the items, depot, and agents and applies the fairness rule from Part 2.7: every item must be deliverable in time, and the target is drawn afterwards.

- `make_episode_set()` produces the *fixed* test games (always the same 200 for a given seed).
- `EpisodeSampler` produces an endless stream of fresh training games.

`env.py` **(the rules of the world).** `GridEnv` runs many games side by side ("vectorized") using NumPy arrays, so 64 games advance in one call. Its three main methods:

- `load(i, spec)` sets game *i* to a new episode.
- `observe(received, cue_mask)` builds the 315-number observation for both agents in every game.
- `step(moves)` applies both agents' moves, pickups, and deliveries, and returns rewards, done flags, and successes.

Everything is deterministic: the same moves always give the same outcome.

`channel.py` **(the radio).** This small file defines what each setup's radio does. Its core function, slightly simplified:

```
def route(sent, mode, vocab_size, rng, perm=None):
    received = sent[:, ::-1].copy()      # scout hears runner, runner hears scout
    if mode == "intact":   return received
    if mode == "silenced": return all "silence" tokens
    if mode == "random":   return random symbols
    if mode == "permuted": return perm[episode, received]   # relabel symbols per episode
```

`cue_mask(condition)` says who sees the target: always the scout, and the runner only in the oracle setup.

3.2 Layer 2: reference agents and the test protocol

`heuristics.py` **(the scripted team).** A hand-written team used only as a reference. Its scout always sends the target's number. Its runner plans shortest paths on the true map and goes for the item the message names, or the nearest item if it has no message.

`rollout.py` **(play a fixed batch of games).** `run_episodes()` plays a list of games to the end with *any* team (scripted or learned) under any radio mode. It returns one record per game (success, steps, the scout's symbols, …) and, if asked, a step-by-step trace for the replay viewer. The training loop does not use it; it exists for testing and replays.

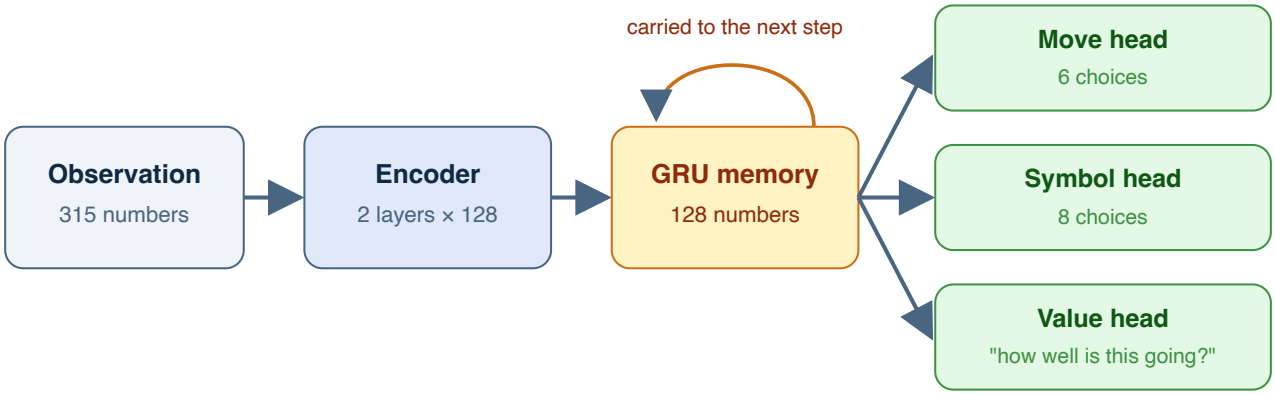
`benchmark.py` **(the official test).** The fixed test protocol lives here, with no learning code at all:

- `EVAL_SEED = 424242` fixes the 200 held-out test games and the 200 training-layout test games. Every run is tested on exactly the same games.
- `eval_plan(condition)` lists what each setup is tested under (comm: intact, permuted, silenced, plus training layouts).
- `summarize(records)` turns 200 game records into per-run numbers: success rate, average reward, steps, and mutual information.

`stats.py` **(statistics).** Bootstrap confidence intervals (explained in Part 4.4) and the mutual-information calculation.

3.3 Layer 3: learning

`policy.py` **(the brain).** `AgentNet` is each agent's network:



- `forward_sequence(obs, h0, first)` runs the network over a sequence of steps. `first` marks where a new game begins, so the memory is wiped there.
- `sample_actions()` draws a move and a symbol from the predicted probabilities and records their joint log-probability, which PPO needs later.

`ppo.py` **(the learning rule).**

- `Rollout` is a container for one batch of experience (observations, actions, rewards, values, …).
- `compute_gae()` turns rewards and value predictions into advantages (Part 1.4).
- `cue_information()` is the bias from Part 1.7: it measures, inside each batch, how much the scout's symbol choices depend on its target cue.
- `ppo_update()` is the PPO step. Its heart is three lines:

```
ratio = (new_logp − old_logp).exp()  # how much has this choice's probability changed?
policy_loss = −min(ratio ∗ adv, ratio.clamp(0.8, 1.2) ∗ adv).mean()  # PPO clipping
loss = policy_loss + 0.5 ∗ value_loss − 0.01 ∗ entropy_bonuses − sig_coef ∗ cue_information
```

`train.py` **(one full training run).** `train()` wires everything together: it creates 64 games, two separate networks with two separate optimizers, and the radio for the chosen setup. It then loops through 1,221 cycles of play, score, and nudge. It writes one row of progress per cycle to `metrics.csv` and saves checkpoints every 250 cycles plus `final.pt` at the end. Section 3.6 walks through one step in detail.

`sweep.py` **(many runs).** Launches the 80 training runs (4 setups × 20 seeds), 6 at a time. It is *resumable*: rerunning it skips any run that already has `final.pt`. Jobs are queued seed by seed.

Parallel jobs can finish out of order; the final audit requires every planned seed and condition.

3.4 Layer 4: measuring and showing

- `evaluate.py` loads each `final.pt`, wraps the two networks in a `LearnedTeam`, and plays the fixed test games under each test condition. It writes `results/per_run.csv` (one row per run and test).
- `analysis.py` averages over the 20 seeds per setup, computes confidence intervals and the comparisons (comm – no comm, intact – permuted, …), checks the pre-registered success criterion, and writes `results/summary.json` and `summary.md`.
- `plots.py` draws the four report figures.
- `replays.py` records step-by-step traces of a few test games (intact, permuted, and no-comm versions of the *same* game) into `viewer/data/replays.js` for the demo viewer.
- `viewer/` is a self-contained web page (HTML + JavaScript, no server needed) that animates those traces and can play each heard symbol as a musical note.
- `report/build_report.py` fills the report template's placeholders (markers wrapped in double curly braces) with numbers from result JSON and typesets it with the pinned CVPR template. This supplementary guide uses the same data loader and a separate HTML-to-PDF renderer.

3.5 Gate scripts

These are small scripts in `scripts/` that check the experiment at each checkpoint:

- `calibrate.py` runs the scripted-team fairness check (Part 2.7).
- `pilot_report.py` scores the pilot runs on training layouts only.
- `check_sweep.py` confirms that all 80 runs finished with equal training budgets.

3.6 The life of one training step

This is what happens, in order, every step inside `train.py`. It runs 156,288 times per training run (128 steps × 1,221 cycles), and each time it advances all 64 games at once: 10 million game steps in total.

1. **Look.** `env.observe(received, mask)` builds each agent's 315 numbers in all 64 games, including the symbol each one heard.
2. **Think and act.** For each agent separately, `forward_sequence()` runs its network (updating its GRU memory), and `sample_actions()` draws a move and a symbol.
3. **The world responds.** `env.step(moves)` moves the agents and handles pickups and deliveries, returning rewards and which games just ended.
4. **The radio carries the symbols.** `route(sent, mode, …)` decides what each agent will hear next step: the partner's symbol, silence, or a random symbol, depending on the setup.
5. **Finished games restart.** Every game that just ended is replaced with a fresh one from `EpisodeSampler`. Its "heard" token is reset to silence, and the `first` flag tells the networks to wipe that game's memory.

6. **Remember.** Observations, actions, log-probabilities, values, rewards, and done flags go into the rollout buffer.

After 128 steps, the cycle ends:

- Compute advantages with GAE.
- Run `ppo_update()` for the scout and then for the runner, each on its own experience: 4 passes over the data in 4 mini-batches.
- Write a row to `metrics.csv`.

3.7 Seeds and exact reproducibility

Every source of randomness is pinned to a number (a **seed**):

Seed	Controls
20260930	the 72 maps
run seed (0–19 for the experiment, 1000+ for pilots)	network starting weights, training games, action sampling
run seed + 1	the random-message radio
424242 (`EVAL_SEED`)	the fixed test games, test-time sampling, and permutations

PyTorch runs single-threaded in deterministic mode. **Bitwise repeatability is checked within the recorded software boundary; it is not guaranteed across hardware or library versions.** A test checks this (`test_training_is_bitwise_reproducible`). It was also confirmed for real: the pilot rerun in the repository reproduced the prototype's learning curve to the last digit.

3.8 What the v1 baseline tests protect

- **The world:** walls block movement, a wrong delivery fails, the time limit works, the observation layout is correct, and replaying the same actions gives the same result.
- **Fairness:** the information-leak test (a no-information runner stays at chance) and the episode rules (every item reachable in time).
- **Learning:** GAE matches a hand calculation, the network's memory resets at episode boundaries, PPO updates change the weights, and training is bit-reproducible.
- **Statistics and reporting:** the bootstrap intervals, mutual information, the success criterion logic, the figures, and the report placeholders.
- **A regression guard** (`test_regression.py`): fixed short-run training and evaluation fingerprints that catch regressions without rerunning the whole benchmark. They do not prove every possible long-run behavior is unchanged.

4. How the experiment was run

4.1 Pre-registration: deciding before looking

Before any of the 80 real training runs started, the plan wrote down exactly what would be measured, how, and what would count as success. That plan was frozen with a git tag (`prereg-v1`). Pre-registration prevents the most common way experiments fool themselves: changing the goalposts after seeing the results. Any later change has to be logged in `docs/DEVIATIONS.md`.

The two pre-registered hypotheses:

- **H1:** learned comm beats no comm by at least 15 percentage points on held-out maps, and the 95% confidence interval excludes zero.
- **H2:** permuting the symbols lowers learned-comm success by at least 10 percentage points, with a 95% confidence interval that excludes zero.

The proposal's success criterion is met only if both hold.

4.2 Gates: go/no-go checkpoints

Gate	What had to be true
G0	The repository exists and installs cleanly.
G1	The world is correct and fair (the scripted-team check passes).
G2	A pilot shows the task is learnable and picks the frozen configuration. Then freeze.
G3	All 80 runs finished with equal budgets.
G4	Results computed, checked, and committed. Verdict recorded.
G5	Report, figures, and demo all build from the results.

4.3 The pilot and the decision rule

The pilot used separate seeds (1000–1002) and judged only on *training* layouts, so it could never peek at the real test. Two configurations ran side by side:

Configuration	Oracle	No comm	Learned comm
Rung 0: plain IPPO	87.2%	20.8%	21.8%
Rung 1: + cue-informativeness bias	91.5%	20.8%	83.2%

A decision table written *before* the pilot said: use plain IPPO if it passes; otherwise use the bias if it passes. Plain IPPO failed the small pilot's selection rule, so the bias configuration was frozen. Note that "no comm" is identical in both rows, down to the last digit. The bias only touches the scout's symbols, which nobody hears in that setup, so it cannot make the control look worse.

Full-budget original-method ablation

A separate 20-seed comparison removes only the cue-information objective, keeping the v1 architecture, environment, optimizer and budget fixed. It was declared after the original outcomes had been seen, so it is post-hoc exploratory and does not extend the frozen hypotheses.

Across 20 full-budget seeds, unbiased IPPO reaches 22.7% (95% CI 21.7–23.6) held-out success, compared with 87.6% (95% CI 86.3–88.6) for cue-directed IPPO. The estimated biased minus unbiased difference is +64.9 pp (95% CI +63.3 to +66.4). The unbiased baseline's intact minus permuted drop is +0.7 pp (95% CI -0.2 to +1.6). The final-checkpoint protocol is retained; method differences bootstrap independent seed sets, while permutation drops pair each policy with itself. The difference estimate is exploratory because the comparison was added after the original outcomes were known; it does not retroactively expand the pre-registered hypotheses.

4.4 Statistics in plain words

Each of the 20 runs per setup gives one success rate (out of 200 test games). The reported number is the **average over the 20 runs**.

The 95% confidence interval (CI) says how sure we are about that average. It is computed with the **bootstrap**:

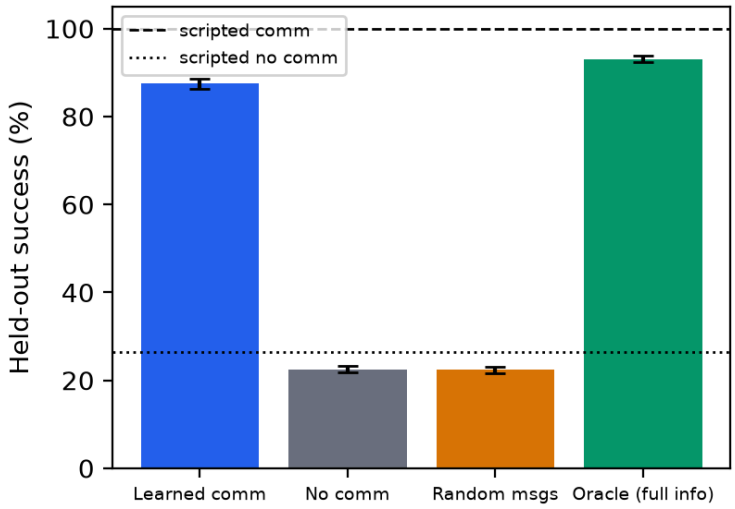
1. Pretend the 20 runs are a bag. Draw 20 runs from it *with replacement* (some get picked twice, some not at all) and compute the average.
2. Do that 10,000 times.
3. The middle 95% of those 10,000 averages is the CI.

A narrow CI means the 20 runs agree closely with each other.

- **Independent comparisons** (comm vs. no comm) resample the two setups separately, because they are different runs.
- **Paired comparisons** (intact vs. permuted) compare each run with *itself*, because it is the same trained pair tested two ways. That is more precise.

5. Reading the results

5.1 Success by setup (Figure 1)

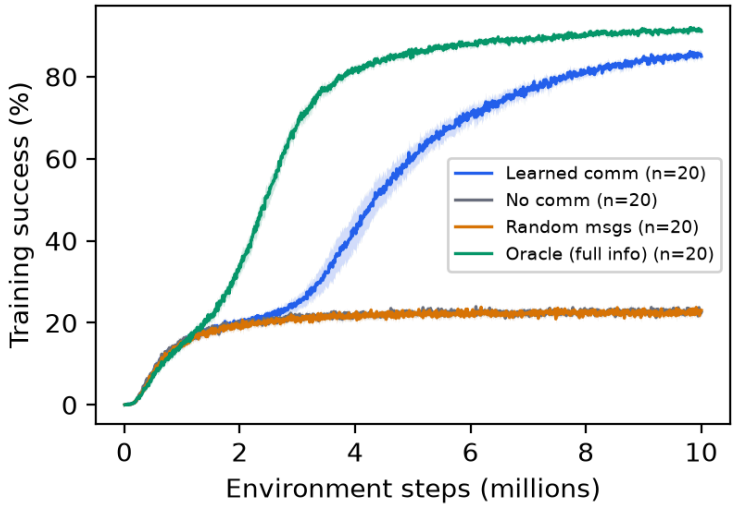


Bar height is the average success over 20 trained pairs on held-out maps; the tiny black bars are the 95% CIs. The dashed line is the scripted team with a fixed codebook; the dotted line is the scripted team guessing.

- Learned comm reaches **87.6%**, close to the oracle's **93.1%** (the runner simply told the answer).
- No comm (**22.5%**) and random messages (**22.3%**) sit at guessing level.
- **H1:** comm − no comm = **+65.0 pp (95% CI +63.5 to +66.3)**. The bar was 15 points.

Why isn't learned comm at 100%? The runner still has to *find* the item in a maze it only partly sees, within 32 steps. Even the oracle, which needs no message at all, reaches only 93.1%.

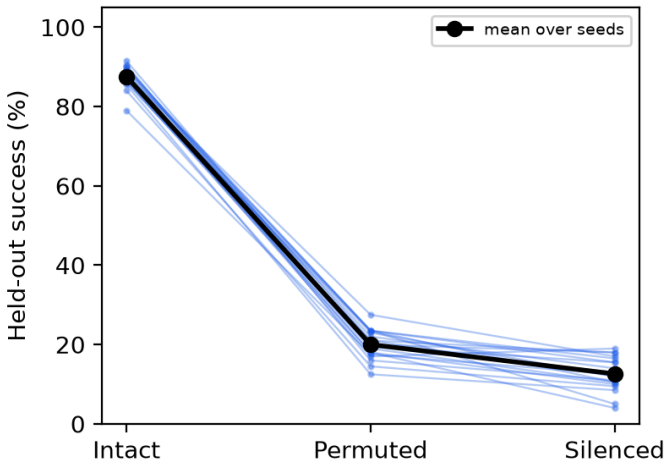
5.2 Learning over time (Figure 2)



Success during training, averaged over the 20 runs per setup (shaded band = 95% range).

- All four setups first learn to deliver *some* item, which takes everyone to about 20% (guessing).
- The oracle takes off first, around 2 million steps: it only has to learn to use the cue it is given.
- Learned comm takes off later, around 3 million steps, because the pair must first *invent* the code and then learn to use it.
- No comm and random messages stay flat at guessing level for the whole run.

5.3 The causal test (Figure 3)

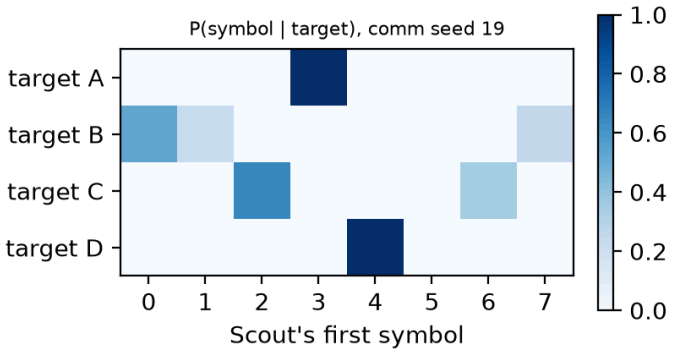


Each thin line is one trained pair, tested on the same 200 held-out games three ways. The thick line is the average.

- **H2:** intact − permuted = **+67.6 pp (95% CI +66.0 to +69.1)**. Every single pair collapses when its symbols are relabeled. This is evidence that the runner depends on stable symbol identities.

- Silenced is even worse: intact − silenced = **+74.9 pp (95% CI +73.0 to +77.0)**. The runners never heard silence after the first step during training, so it confuses them more than a wrong symbol does.

5.4 The invented dictionary (Figure 4)



For one typical pair (the median seed): how often the scout's first symbol was each number (columns), for each target (rows). Dark = always.

How to read it: in this pair, target A was always signalled with symbol 3, and D always with symbol 4. B and C each used a few **synonyms** (B was sometimes 0, 1, or 7), but **no symbol was ever shared between two targets**, so the runner can always decode it. Other pairs invented completely different assignments. That is what "meanings were never programmed" looks like.

5.5 What mutual information does and doesn't show

Mutual information (MI) measures how much the scout's first symbol tells you about the target, from 0 bits (nothing) to 2 bits (everything). It is 1.98 bits for learned comm, but also 1.99 bits for no comm, where nobody hears the scout at all. The bias makes every scout informative. That's why MI alone can't prove communication: it measures the *speaker*. Only the permutation test measures whether the *listener* uses the message. This is a known pitfall in emergent-communication research (Lowe et al., 2019), and the project's design handles it.

6. The vocal channel (bonus experiment)

This experiment was declared as *exploratory* before pre-registration. It does not count toward H1 or H2.

6.1 What changed

- **A voice instead of beeps.** Each agent emits a continuous "pitch" between −1 and 1 instead of one of 8 symbols. The listener hears it with random noise added (standard deviation 0.1). The network's symbol head becomes a pitch head that outputs a mean pitch, with a single learned spread, and the actual pitch is sampled from a bell curve around the mean.
- **An ear like a cochlea.** The listener hears through 8 overlapping "tuning curves" spread across the pitch range, the way the inner ear has hair cells tuned to different frequencies. Each curve responds most to pitches near its center. This keeps the signal continuous and noisy, but makes it learnable.
- **A continuous version of the bias.** It rewards the scout for making the four targets' pitches *distinct and consistent* using a Gaussian pairwise-distance information surrogate inspired by Kolchinsky and Tracey (2017). Clipping means it is not an exact bound for the actual heard channel; see the report reference.
- **A different causal test: shuffling.** Each runner hears the pitch spoken in a *different* test game. It still sounds like a scout, but no longer matches this game's target.
- **Longer training.** The vocal arm uses 20.0 million steps per run. Its larger budget means cross-arm success rates do not isolate channel type.

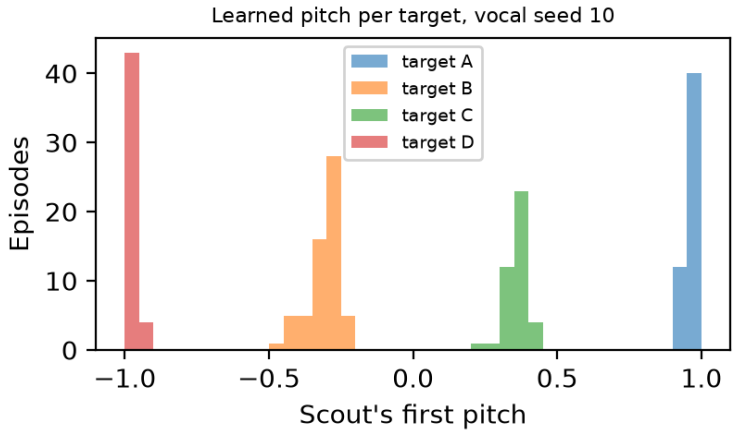
6.2 Why the first design failed

The first version used a raw pitch number as the ear and squashed pitches with a curve that saturates at the edges. Two things went wrong:

- The scout's pitches jammed at the extremes (+1 and −1), and two targets flipped back and forth between them.
- The runner never learned to use the signal: shuffling made no difference.

The fixes were the cochlea-like ear, a pitch that does not saturate, and a bias that penalizes inconsistent notes. This redesign was done on pilot seeds, before any real vocal runs, and is documented in the spec.

6.3 Results



For one typical vocal pair: the scout's first pitch in each held-out test game, colored by target. Four separate peaks mean the scout invented four distinct "notes".

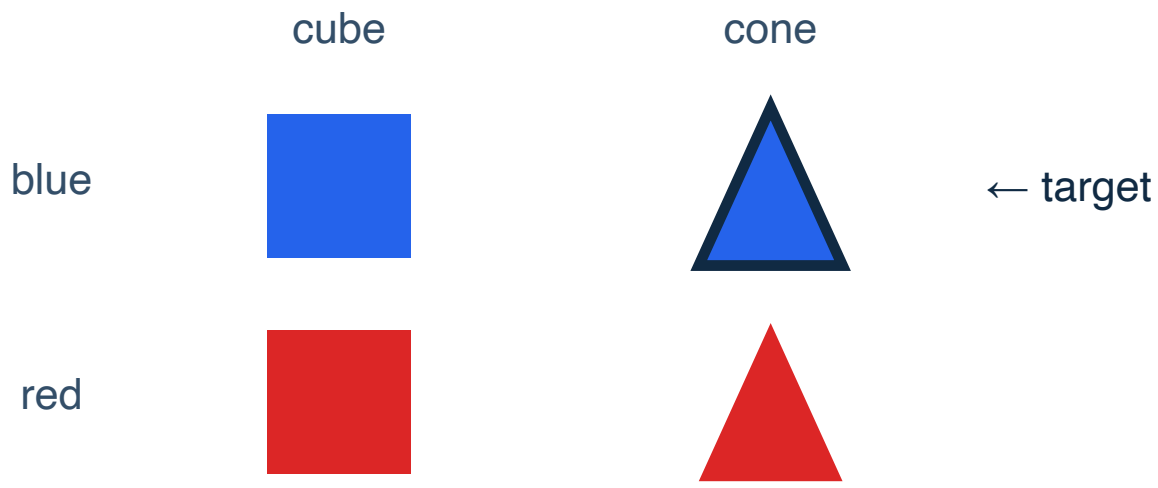
- Vocal comm reaches **73.0%** vs. **21.4%** without communication: a gain of +51.5 pp (95% CI +48.3 to +54.6).
- Intact − shuffled = +57.9 pp (95% CI +54.5 to +61.0): hearing another game's voice wipes out the gain, so the runner listens to the actual pitch.
- The scout split its continuous voice into four notes, roughly evenly spaced, illustrating separated pitch categories in that representative run; no optimal arrangement is established.

7. v2: words that combine

v2 is a second, separately pre-registered experiment (tag `prereg-v2`). v1 asked whether the agents could invent a useful code. v2 asks whether parts of a two-word message line up with parts of a meaning, and whether the learned protocol works for combinations the agents never encountered in training.

7.1 What changed

- **Items have two properties.** Five colors × five shapes give 25 kinds. The scout's secret is now something like “blue cone.”
- **Every episode contains look-alikes.** The four items form a 2×2 square: two colors × two shapes. On ordinary seen-square sampling, knowing only “blue” leaves two candidates, and knowing only “cone” also leaves two.



One episode's four items. “Blue” leaves two candidates; “cone” leaves two; “blue cone” leaves one.

- **Messages have two words.** Each word is one of eight symbols, giving 64 possible messages. That is enough to give all 25 targets unrelated names, so the network is not forced to put color in one position and shape in the other.
- **Five combinations are absent from training:** red sphere, yellow ring, green cylinder, blue cube, purple cone. Every color and every shape still appears in other combinations.
- **The four controls remain.** Learned comm receives the scout's words; no comm receives silence; random messages receive unrelated words; and the oracle runner sees the target. Each learned condition trains for 20.0 M environment steps, with weight 0.2 on the sum of each word's information about the joint color–shape cue. No fixed word-to-property assignment or penalty for redundancy is supplied.
- **The tests separate maps from meanings.** `held_out` uses new layouts with seen combinations. `unseen` uses the five combinations absent from training. `train` checks familiar layouts.

7.2 How the new tests work

There are four message interventions for learned comm:

- **Whole-message permutation** relabels both words and tests whether the runner depends on their learned identities.
- **Word-1 and word-2 permutation** scramble one position at a time. A larger drop for one position shows where the listener is sensitive, but it does not by itself prove that the position means color or shape.
- **Silencing** removes the message on the pre-registered splits where this intervention is run.

The analysis also records color and shape accuracy among picks, and divides wrong picks into same-color, same-shape, and opposite-property twins. The scout's first messages get four further descriptions:

- mutual information between each word and each property;
- **topographic similarity**, which asks whether similar meanings have similar messages;
- **positional disentanglement**, which asks whether word positions specialize in different properties; and
- a lexicon plot showing the median seed's symbol frequencies for every color and shape.

These measurements can be consistent with a compositional code when they agree. None is a categorical proof on its own.

7.3 The fixed sequence

The pre-registration tests claims in the order A → B → C. If an earlier claim is not supported or cannot be estimated, later claims are not reached.

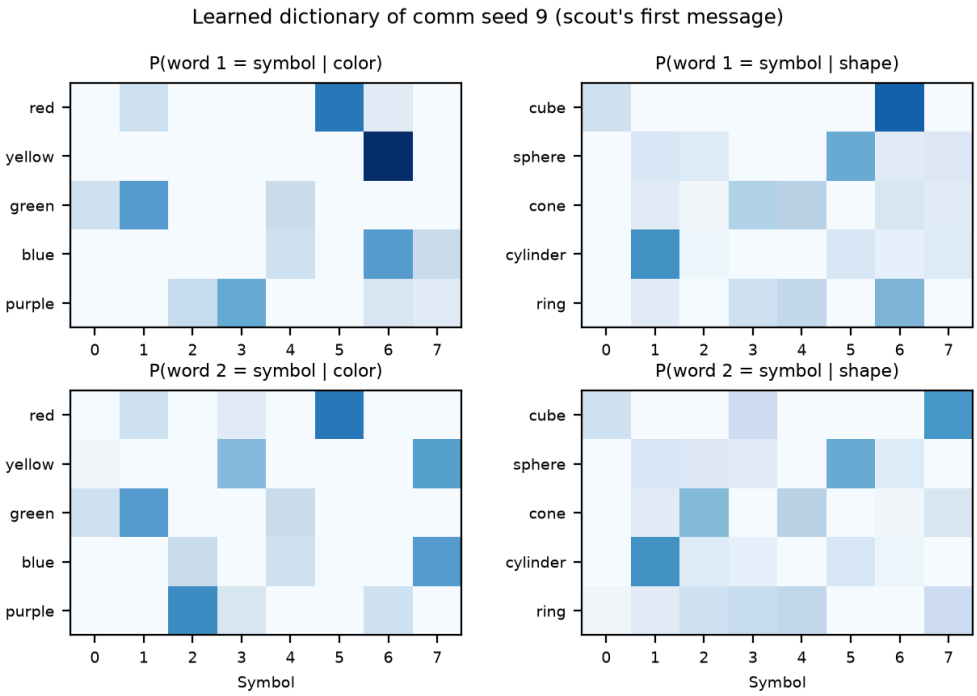
Claim	What must happen	Result
A: communication works in v2	H1: seen comm gain; H2: seen whole-message permutation drop	supported
B: the language generalizes	A, then H3: unseen comm gain	not supported
C: unseen success above the fixed 50% reference	B, then H4: unseen success and its CI above 50%	not reached

The corresponding estimates are H1 +15.3 pp (95% CI +13.0 to +17.6), H2 +22.3 pp (95% CI +20.2 to +24.3), H3 +7.0 pp (95% CI +4.1 to +10.2), and H4 26.7% (95% CI 23.9–29.8). If an input is JSON `null`, the guide prints “not estimable” instead of inventing a value.

7.4 Reading the results

Condition	Seen combinations	Never-seen combinations
Learned comm	37.1% (95% CI 35.1–39.0)	26.7% (95% CI 23.9–29.8)
No comm	21.8% (95% CI 20.7–22.9)	19.7% (95% CI 18.8–20.6)
Random messages	20.5% (95% CI 19.8–21.3)	18.4% (95% CI 17.6–19.2)
Oracle	84.6% (95% CI 83.8–85.5)	73.3% (95% CI 70.8–75.7)

The word-specific success drops are +10.0 pp (95% CI +8.4 to +11.5) for word 1 and +10.2 pp (95% CI +8.4 to +11.9) for word 2 on seen combinations; intact − silenced is +24.6 pp (95% CI +22.2 to +27.0). Among the runner's seen-combination picks, color is correct 68.2% and shape is correct 71.1%. These are descriptive measures and should be read with the lexicon rather than treated as a separate hypothesis.



The median comm seed's learned lexicon. Compare each word's color and shape bands; an apparent alignment is descriptive evidence, not a verdict by itself.

For the learned comm condition on seen combinations, topographic similarity is 0.24 and positional disentanglement is 0.16. Values nearer 1 indicate stronger structure under those definitions, but neither number proves compositionality.

7.5 Why 50% is not an unseen ceiling

The unseen test has a special prior: its five target pairs form a one-to-one matching. A source-only enumeration, using no learned model or evaluation result, lists 80 equally likely target–square states. In 60 states the square contains one held-out candidate; in 20 it contains two. Choosing from that support

with uniform tie-breaking identifies the target before navigation at 87.5% in expectation. Knowing either property plus the matching identifies it at 100.0%. This is a theoretical identification calculation, not learned performance or navigation success.

That is why Claim C means only “unseen success above the fixed 50.0% reference.” It does not mean that both properties transferred, and it does not prove compositionality. Word interventions, the lexicon, mutual information, topographic similarity, and positional disentanglement provide descriptive context.

Generated interpretation. Claim A was supported, but Claim B was not, so Claim C was not reached. The experiment meets the seen-combination communication criterion but not the pre-registered generalization criterion. The unseen estimates and structural metrics remain descriptive.

8. Try it yourself

All commands run from the project folder. `uv` manages Python and the libraries.

```
uv sync                      # install everything (first time only)
uv run pytest -q             # run the full test suite
open viewer/index.html       # the demo: click a tab, press Play, turn Sound on
open viewer/vocal.html       # the vocal-channel demo
uv run python scripts/calibrate.py   # rerun the fairness check (Part 2.7)
```

Watch an agent learn (about 10 minutes). Train one pair on a short budget, in a scratch folder that is gitignored:

```
uv run python -m emcomm.train --config configs/base.toml --condition comm \
    --seed 1234 --out runs/scratch/comm --total-env-steps 4005888
cut -d, -f2,4 runs/scratch/comm/metrics.csv | tail -5    # env steps, success rate
```

The success rate should creep up from near 0 to about 20% (guessing), then start rising past it around 3–4 million steps.

Ideas to explore (always with a copy of the config, never by editing `configs/base.toml`, which is frozen):

- Train `no_comm` with the same seed and compare the curves.
- Make a copy of the config with `sig_coef = 0.0` (no bias), to see the chicken-and-egg problem for yourself.
- Change `vocab_size` to 4 or 16 and see whether the code still forms.

9. Glossary

Term	Meaning
Advantage	How much better (or worse) an action turned out than the agent expected.
Agent	A learner that observes, acts, and receives reward. Here: the scout and the runner.
Bootstrap	Resampling the runs many times to estimate how uncertain an average is.
Checkpoint (`.pt`)	A saved copy of the networks' weights.
Compositional language	A language whose message parts stand for parts of the meaning, so known parts can be combined in new ways.
Confidence interval (CI)	An interval estimating uncertainty; 95% describes the procedure's repeated-sample coverage.
Cue	The target information given to the scout.
Emergent communication	Message assignments and use learned without a programmed dictionary; auxiliary biases must be disclosed.
Entropy bonus	A small reward for keeping some randomness, so agents keep exploring.
Episode	One game, from start to delivery or timeout.
GAE	Generalized Advantage Estimation, a standard way to compute advantages.
GRU	A recurrent network block that gives the agent short-term memory.
Held-out layouts	Maps never used in training, kept for the final test.
IPPO	Independent PPO: each agent learns with its own separate network.
Mutual information (MI)	How much one thing tells you about another, in bits.
Oracle	A learned reference where the runner is told the target.
Percentage points (pp)	The difference between two percentages (for example, 80% − 30% = 50 pp).
Permutation test	Relabeling the symbols to check that their meaning matters.
Policy	The agent's decision rule, here a neural network.
Positional disentanglement	How much each word position specializes in one property (1 means each position encodes exactly one).
PPO	Proximal Policy Optimization, the learning recipe used here.
Pre-registration	Writing down the hypotheses and analysis before seeing the results.
Seed	A number controlling randomness within the recorded execution boundary.
Topographic similarity	How well distances between messages track distances between meanings (1 means perfectly).
Weights	The adjustable numbers inside a neural network.

10. Questions your professor might ask

"Did you program what the symbols mean?" No. Nothing in the code links any symbol to any item. The symbol assignments are learned separately in each training run. The bias only rewards the scout for saying *something different* per item; it never says what.

"How do you know they're really communicating, not something else?" Three pieces of evidence:

- Their observations omit other-agent positions and the uncued runner's target cue. The scout cannot reach the items, so messages carry target information in the base task.
- Random messages provide a matched control without useful target information.
- Relabeling the symbols substantially reduces aggregate success (intact – permuted = +67.6 pp (95% CI +66.0 to +69.1)).

"Isn't the bias cheating?" It is a disclosed design choice informed by a small train-only pilot. The scout receives a cue-information objective, while the listener learns from task reward. The direct ablation isolates this added term within the selected task and configuration. Positive signalling is prior work (Eccles et al., 2019).

"Why isn't it 100%?" Learned comm reaches 87.6%. The runner sees only a 7×7 window and has 32 steps. Even when told the target directly (the oracle), it reaches 93.1%. The oracle is a learned reference, not a guaranteed upper limit.

"Why such a simple world?" Because the goal was a *clean causal test*. A small world makes it possible to block every other channel, test the information path needed for reliable coordination, and run 80 independent trainings with proper statistics. A 3D world or a harder task would be a natural next step.

"What surprised you?" The small plain-IPPO pilot failed the selection rule, while the biased method communicated reliably in this task. That does not establish a universal necessity for the bias. Silence also reduced success more than permutation, consistent with a message-absence distribution shift.